

Saving changes in Git

When working in Git, or other version control systems, the concept of "saving" is a more nuanced process than saving in a word processor or other traditional file editing applications. The traditional software expression of "saving" is synonymous with the Git term "committing". A commit is the Git equivalent of a "save". Traditional saving should be thought of as a file system operation that is used to overwrite an existing file or write a new file. Alternatively, Git committing is an operation that acts upon a collection of files and directories.

Saving changes in Git vs SVN is also a different process. SVN Commits or 'check-ins' are operations that make a remote push to a centralized server. This means an SVN commit needs Internet access in order to fully 'save' project changes. Git commits can be captured and built up locally, then pushed to a remote server as needed using the `git push -u origin main` command. The difference between the two methods is a fundamental difference between architecture designs. Git is a distributed application model whereas SVN is a centralized model. Distributed applications are generally more robust as they do not have a single point of failure like a centralized server.

The commands: `git add`, `git status`, and `git commit` are all used in combination to save a snapshot of a Git project's current state.

Git has an additional saving mechanism called 'the stash'. The stash is an ephemeral storage area for changes that are not ready to be committed. The stash operates on the working directory, the first of [the three trees](#) and has extensive usage options. To learn more visit the [git stash](#) page.

A Git repository can be configured to ignore specific files or directories. This will prevent Git from saving changes to any ignored content. Git has multiple methods of configuration that manage the ignore list. Git ignore configure is discussed in further detail on the [git ignore](#) page.

git add

The `git add` command adds a change in the working directory to the staging area (staging area (also called the **index**) is an *intermediate space where changes are gathered before they are committed*). It tells Git that you want to include updates to a particular file in the next commit. However, `git add` doesn't really affect the repository in any significant way—changes are not actually recorded until you run `git commit`.

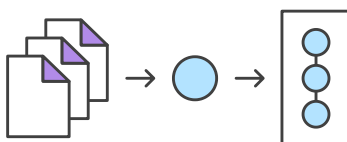
In conjunction with these commands, you'll also need [git status](#) to view the state of the working directory and the staging area.

How it works

The `git add` and `git commit` commands compose the fundamental Git workflow. These are the two commands that every Git user needs to understand, regardless of their team's collaboration model. They are the means to record versions of a project into the repository's history.

Developing a project revolves around the basic edit/stage/commit pattern. First, you edit your files in the working directory. When you're ready to save a copy of the current state of the project, you stage changes with `git add`. After you're happy with the staged snapshot, you commit it to the project history with `git commit`. The `git reset` command is used to undo a commit or staged snapshot.

In addition to `git add` and `git commit`, a third command `git push` is essential for a complete collaborative Git workflow. `git push` is utilized to send the committed changes to remote repositories for collaboration. This enables other team members to access a set of saved changes.



The `git add` command should not be confused with `svn add`, which adds a file to the repository. Instead, `git add` works on the more abstract level of changes. This means that `git add` needs to be called every time you alter a file, whereas `svn add` only needs to be called once for each file. It may sound redundant, but this workflow makes it much easier to keep a project organized.

The staging area

Instead of committing all of the changes you've made since the last commit, the stage lets you group related changes into highly focused snapshots before actually committing it to the project history. This means you can make all sorts of edits to unrelated files, then go back and split them up into logical commits by adding related changes to the stage and commit them piece-by-piece. As in any revision control system, it's important to create atomic commits so that it's easy to track down bugs and revert changes with minimal impact on the rest of the project.

Common options

```
1 git add <file>
```

Stage all changes in `<file>` for the next commit.

```
1 git add <directory>
```

Stage all changes in `<directory>` for the next commit.

```
1 git add -p
```

Begin an interactive staging session that lets you choose portions of a file to add to the next commit. This will present you with a chunk of changes and prompt you for a command. Use `y` to stage the chunk, `n` to ignore the chunk, `s` to split it into smaller chunks, `e` to manually edit the chunk, and `q` to exit.

Examples

When you're starting a new project, `git add` serves the same function as `svn import`. To create an initial commit of the current directory, use the following two commands:

```
1 git add .
2 git commit
```

Once you've got your project up-and-running, new files can be added by passing the path to `git add`:

```
1 git add hello.py
2 git commit
```

The above commands can also be used to record changes to existing files. Again, Git doesn't differentiate between staging changes in new files vs. changes in files that have already been added to the repository.